

Nery Javier de la Cruz Huinil – 261233

David Maximiliano Pablo Cuy - 261341

Análisis

Requisitos funcionales

Información que el sistema debe conservar

- De cada vehículo hay que guardar la placa (que no se puede repetir), la marca, el modelo, la tarifa diaria, en qué estado está (Disponibile, Alquilado o En mantenimiento) y cuántos días lleva alquilado desde su último mantenimiento. Además, cada tipo tiene lo suyo: el automóvil guarda cuántos pasajeros caben y si es automático, la moto su cilindraje, la camioneta cuántas toneladas aguanta y el microbús sus pasajeros y si trae piloto.
- De los clientes guardo su identificador (que tampoco se repite), su nombre, sus licencias (A, B, C o M) y cuántos alquileres ha confirmado y cuántos tiene activos. Si es individual se identifica con su DPI de 13 dígitos, y si es corporativo con su NIT, el nombre de la empresa y un contacto.
- De cada alquiler guardo su número correlativo, el cliente, el vehículo, los días, el subtotal, el descuento, el total y si sigue activo. Y la empresa lleva la cuenta de lo que ha ganado en total, lo que ha ganado por categoría y cuánto ha dado en descuentos.

Operaciones que debe permitir

- El sistema tiene que dejar registrar vehículos y clientes, ver la flota y los clientes, cotizar, confirmar o cancelar alquileres, registrar devoluciones y fines de mantenimiento, y sacar los reportes (flota por categoría y estado, ingresos, descuentos, alquileres activos e historial de un cliente).

Reglas de negocio

- Cobro: el subtotal es la tarifa por los días más el recargo. El automóvil automático paga Q50 extra por día, la moto de más de 250 cc paga Q75 una sola vez, la camioneta paga Q100 por tonelada por día y el microbús con piloto paga Q250 extra por día.
- Descuento: el cliente individual no tiene descuento en sus primeros tres alquileres y a partir del cuarto se le da 5 %. El corporativo siempre tiene 10 %. El total es el subtotal menos el descuento.
- Licencias: la A también sirve para B y C, la B sirve para C y la M va por aparte. El automóvil pide C o más, la moto pide M, la camioneta y el microbús sin piloto piden B o más, y el microbús con piloto no pide nada porque maneja el piloto de la empresa.
- El individual solo puede tener 1 alquiler activo y el corporativo hasta 3. Los vehículos se van a mantenimiento al llegar a 30 días (automóvil), 20 (moto), 15 (camioneta) y 25 (microbús).

Condiciones de validez

- Para registrar: la placa o el identificador no pueden ir vacíos ni repetidos, la tarifa, los pasajeros, el cilindraje y la capacidad tienen que ser mayores que cero, el DPI debe tener 13 dígitos y el cliente necesita al menos una licencia.
- Para cotizar: la placa y el cliente tienen que existir y los días deben ser un entero positivo. Se puede cotizar aunque el vehículo no esté disponible, y ahí se muestran todas las razones por las que no se podría alquilar.
- Para alquilar: el vehículo tiene que estar disponible, el cliente debe tener la licencia correcta y no haber llegado a su límite. Si cancela, no cambia nada. Para devolver, el vehículo tiene que estar alquilado, y para terminar el mantenimiento, tiene que estar en mantenimiento.

Clases y responsabilidades

- Vehiculo es abstracta y tiene todo lo que comparten los vehículos: guarda los datos comunes, maneja el estado y calcula el subtotal. VehiculoPasajeros también es abstracta y solo guarda los pasajeros. Automovil, Motocicleta, CamionetaCarga y Microbus ponen las reglas de cada categoría.

- Cliente es abstracta y guarda lo que tienen en común los clientes y sus conteos. ClienteIndividual y ClienteCorporativo deciden cada uno su descuento y su límite.
- Cotizacion calcula cuánto saldría y por qué no se podría alquilar, sin cambiar nada. Alquiler guarda un alquiler ya confirmado. RentaMovil maneja las listas, revisa que las operaciones sean válidas y hace los reportes. Main tiene el menú y el método main. TipoLicencia y EstadoVehiculo los hice como enums.

Todos los objetos reciben sus datos por el constructor, que los revisa y lanza IllegalArgumentException si algo está mal. Cada vehículo nuevo empieza Disponible con 0 días y cada cliente con 0 alquileres. RentaMovil empieza vacía con el correlativo en 1, y Main carga los datos iniciales usando los mismos métodos de registro.

Propiedades

#	tipoDato	nombre	visibilidad	motivo	Clase
1	EstadoVehiculo	DISPONIBLE, ALQUILADO, EN_MANTENIMIENTO	public static final	Constantes del enum; limitan el estado a tres valores válidos.	EstadoVehiculo
2	TipoLicencia	A, B, C, M	public static final	Constantes del enum que representan los tipos de licencia de la política.	TipoLicencia
3	String	placa	private	Identificador único del vehículo; no puede repetirse ni estar vacío.	Vehiculo
4	String	marca	private	Marca del vehículo para consultas y cotizaciones.	Vehiculo
5	String	modelo	private	Modelo del vehículo para consultas y cotizaciones.	Vehiculo
6	double	tarifaDiaria	private	Tarifa base por día (mayor que cero); base de todo subtotal.	Vehiculo
7	EstadoVehiculo	estado	private	Estado actual; solo cambia mediante los métodos de la clase.	Vehiculo
8	int	diasDesdeMantenimiento	private	Días alquilados acumulados desde el último mantenimiento; se compara con el umbral.	Vehiculo
9	int	cantidadPasajeros	private	Capacidad de pasajeros (mayor que cero); dato común a automóviles y microbuses.	VehiculoPasajeros

10	boolean	transmisionAutomatica	private	Indica si es automático; determina el recargo de Q50 por día.	Automovil
11	double	RECARGO_AUTOMATICO	private static final	Constante Q50 por día; evita valores literales en el cálculo.	Automovil
12	int	UMBRAL_MANTENIMIENTO	private static final	Constante de 30 días para enviar a mantenimiento.	Automovil
13	int	cilindraje	private	Cilindraje en cc (mayor que cero); determina el recargo.	Motocicleta
14	int	LIMITE_CILINDRAJE	private static final	Constante 250 cc; a partir de ahí aplica recargo.	Motocicleta
15	double	RECARGO_ALTO_CILINDRAJE	private static final	Constante Q75 por todo el alquiler.	Motocicleta
16	int	UMBRAL_MANTENIMIENTO	private static final	Constante de 20 días para enviar a mantenimiento.	Motocicleta
17	double	capacidadToneladas	private	Capacidad máxima en toneladas (admite decimales, mayor que cero).	CamionetaCarga
18	double	RECARGO_POR_TONELADA	private static final	Constante Q100 por tonelada por día.	CamionetaCarga
19	int	UMBRAL_MANTENIMIENTO	private static final	Constante de 15 días para enviar a mantenimiento.	CamionetaCarga
20	boolean	incluyePiloto	private	Característica fija del microbús; define recargo y licencia requerida.	Microbus
21	double	RECARGO_PILOTO	private static final	Constante Q250 por día cuando incluye piloto.	Microbus
22	int	UMBRAL_MANTENIMIENTO	private static final	Constante de 25 días para enviar a mantenimiento.	Microbus
23	String	identificador	private	DPI o NIT; identifica al cliente y no puede repetirse.	Cliente
24	String	nombre	private	Nombre de la persona o de la empresa.	Cliente
25	ArrayList<TipoLicencia>	licencias	private	Licencias presentadas (al menos una); se usan para validar el vehículo.	Cliente
26	int	alquileresConfirmados	private	Total de alquileres confirmados; define el descuento del cliente individual.	Cliente
27	int	alquileresActivos	private	Alquileres en curso; se compara con el límite del tipo de cliente.	Cliente

28	int	LIMITE_ACTIVOS	private static final	Constante 1: solo un alquiler activo a la vez.	ClienteIndividual
29	int	ALQUILERES_SIN_DESCUENTO	private static final	Constante 3: alquileres confirmados sin descuento.	ClienteIndividual
30	double	PORCENTAJE_DESCUENTO	private static final	Constante 0.05 (5 %) desde el cuarto alquiler.	ClienteIndividual
31	String	nombreContacto	private	Persona de contacto de la empresa.	ClienteCorporativo
32	int	LIMITE_ACTIVOS	private static final	Constante 3: hasta tres alquileres activos.	ClienteCorporativo
33	double	PORCENTAJE_DESCUENTO	private static final	Constante 0.10 (10 %) siempre.	ClienteCorporativo
34	Vehiculo	vehiculo	private	Vehículo que se cotiza (tipo común; puede ser cualquier categoría).	Cotizacion
35	Cliente	cliente	private	Cliente que solicita la cotización (tipo común).	Cotizacion
36	int	dias	private	Días del alquiler (entero positivo).	Cotizacion
37	double	subtotal	private	Tarifa × días + recargo de la categoría.	Cotizacion
38	double	descuento	private	Descuento calculado por el tipo de cliente.	Cotizacion
39	double	total	private	Subtotal menos descuento.	Cotizacion
40	ArrayList<String>	razonesRechazo	private	Todas las razones por las que no podría alquilarse; vacía si es válido.	Cotizacion
41	int	numero	private	Número correlativo del alquiler.	Alquiler
42	Cliente	cliente	private	Cliente que realizó el alquiler; permite armar su historial.	Alquiler
43	Vehiculo	vehiculo	private	Vehículo alquilado; permite localizar el alquiler al devolver.	Alquiler
44	int	dias	private	Días alquilados; se suman al acumulado al devolver.	Alquiler
45	double	subtotal	private	Subtotal cobrado al confirmar.	Alquiler
46	double	descuento	private	Descuento aplicado al confirmar.	Alquiler
47	double	total	private	Monto pagado; se suma a los ingresos.	Alquiler
48	boolean	activo	private	true mientras el vehículo no se devuelva.	Alquiler
49	ArrayList<Vehiculo>	flota	private	Única colección de vehículos, declarada con el tipo común.	RentaMovil
50	ArrayList<Cliente>	clientes	private	Única colección de clientes, declarada con el tipo común.	RentaMovil

51	ArrayList<Alquiler>	alquileres	private	Historial de todos los alquileres confirmados.	RentaMovil
52	int	siguienteNumeroAlquiler	private	Contador para asignar el número correlativo.	RentaMovil
53	double	ingresosTotales	private	Dinero acumulado por alquileres confirmados.	RentaMovil
54	double	descuentosTotales	private	Monto total de descuentos otorgados.	RentaMovil
55	HashMap<String, Double>	ingresosPorCategoria	private	Ingresos agrupados por la categoría que reporta cada vehículo.	RentaMovil
56	Scanner	scanner	private static	Lector único de la consola.	Main
57	RentaMovil	empresa	private static	Instancia que administra toda la información durante la ejecución.	Main

Métodos

#	retorno	nombre	visibilidad	motivo	Clase
1	boolean	autoriza(TipoLicencia requerida)	public	Indica si esta licencia cubre la requerida (A cubre B y C; B cubre C; M solo M).	TipoLicencia
2	—	Vehiculo(String placa, String marca, String modelo, double tarifaDiaria)	protected	Constructor: valida placa no vacía y tarifa > 0; deja el estado DISPONIBLE y el acumulado en 0.	Vehiculo
3	String	getPlaca()	public	Lectura de la placa para búsquedas.	Vehiculo
4	String	getMarca()	public	Lectura de la marca.	Vehiculo
5	String	getModelo()	public	Lectura del modelo.	Vehiculo
6	double	getTarifaDiaria()	public	Lectura de la tarifa diaria.	Vehiculo
7	EstadoVehiculo	getEstado()	public	Lectura del estado para cotizaciones y reportes.	Vehiculo
8	int	getDiasDesdeMantenimiento()	public	Lectura del acumulado de días.	Vehiculo
9	boolean	estaDisponibile()	public	Devuelve true si el estado es DISPONIBLE.	Vehiculo
10	double	calcularSubtotal(int dias)	public final	Devuelve tarifaDiaria × dias + calcularRecargo(dias). Es final para que la fórmula base sea igual para todos.	Vehiculo
11	double	calcularRecargo(int dias)	protected abstract	Recargo propio de la categoría; cada subclase lo implementa.	Vehiculo
12	TipoLicencia	getLicenciaRequerida()	public abstract	Licencia mínima que exige la categoría.	Vehiculo

13	boolean	aceptaLicencias(ArrayList<TipoLicencia> licencias)	public	Devuelve true si alguna licencia del cliente autoriza la requerida.	Vehiculo
14	int	getUmbralMantenimiento()	public abstract	Días acumulados que envían el vehículo a mantenimiento.	Vehiculo
15	String	getCategoria()	public abstract	Nombre de la categoría para mostrar y agrupar reportes.	Vehiculo
16	String	describirCaracteristicas()	public abstract	Texto con los datos propios de la categoría.	Vehiculo
17	void	marcarAlquilado()	public	Cambia el estado a ALQUILADO; lanza IllegalStateException si no está disponible.	Vehiculo
18	boolean	registrarDevolucion(int dias)	public	Suma los días al acumulado; si alcanza el umbral pasa a EN_MANTENIMIENTO, si no a DISPONIBLE. Devuelve true si entró a mantenimiento.	Vehiculo
19	void	finalizarMantenimiento()	public	Regresa el vehículo a DISPONIBLE y el acumulado a 0; lanza excepción si no estaba en mantenimiento.	Vehiculo
20	void	cargarDiasAcumulados(int dias)	public	Registra días previos (≥ 0) para los datos iniciales; no cambia el estado.	Vehiculo
21	String	toString()	public	Sobrescribe Object: datos comunes, estado y describirCaracteristicas().	Vehiculo
22	—	VehiculoPasajeros(String placa, String marca, String modelo, double tarifaDiaria, int cantidadPasajeros)	protected	Constructor: llama a super y valida pasajeros > 0.	VehiculoPasajeros
23	int	getCantidadPasajeros()	public	Lectura de la capacidad de pasajeros.	VehiculoPasajeros
24	—	Automovil(String placa, String marca, String modelo, double tarifaDiaria, int cantidadPasajeros, boolean transmisionAutomatica)	public	Constructor del automóvil.	Automovil
25	boolean	isTransmisionAutomatica()	public	Lectura del tipo de transmisión.	Automovil
26	double	calcularRecargo(int dias)	protected	@Override: $Q50 \times \text{días}$ si es automático; 0 si es manual.	Automovil
27	TipoLicencia	getLicenciaRequerida()	public	@Override: devuelve C.	Automovil
28	int	getUmbralMantenimiento()	public	@Override: devuelve 30.	Automovil
29	String	getCategoria()	public	@Override: devuelve "Automóvil".	Automovil

30	String	describirCaracteristicas()	public	@Override: pasajeros y transmisión.	Automovil
31	—	Motocicleta(String placa, String marca, String modelo, double tarifaDiaria, int cilindraje)	public	Constructor: valida cilindraje > 0.	Motocicleta
32	int	getCilindraje()	public	Lectura del cilindraje.	Motocicleta
33	double	calcularRecargo(int dias)	protected	@Override: Q75 una sola vez si el cilindraje es mayor que 250 cc; 0 en otro caso.	Motocicleta
34	TipoLicencia	getLicenciaRequerida()	public	@Override: devuelve M.	Motocicleta
35	int	getUmbralMantenimiento()	public	@Override: devuelve 20.	Motocicleta
36	String	getCategoria()	public	@Override: devuelve "Motocicleta".	Motocicleta
37	String	describirCaracteristicas()	public	@Override: cilindraje.	Motocicleta
38	—	CamionetaCarga(String placa, String marca, String modelo, double tarifaDiaria, double capacidadToneladas)	public	Constructor: valida capacidad > 0.	CamionetaCarga
39	double	getCapacidadToneladas()	public	Lectura de la capacidad máxima.	CamionetaCarga
40	double	calcularRecargo(int dias)	protected	@Override: Q100 × toneladas × días.	CamionetaCarga
41	TipoLicencia	getLicenciaRequerida()	public	@Override: devuelve B.	CamionetaCarga
42	int	getUmbralMantenimiento()	public	@Override: devuelve 15.	CamionetaCarga
43	String	getCategoria()	public	@Override: devuelve "Camioneta de carga".	CamionetaCarga
44	String	describirCaracteristicas()	public	@Override: capacidad en toneladas.	CamionetaCarga
45	—	Microbus(String placa, String marca, String modelo, double tarifaDiaria, int cantidadPasajeros, boolean incluyePiloto)	public	Constructor del microbús.	Microbus
46	boolean	isIncluyePiloto()	public	Lectura de si se entrega con piloto.	Microbus
47	double	calcularRecargo(int dias)	protected	@Override: Q250 × días si incluye piloto; 0 si no.	Microbus
48	TipoLicencia	getLicenciaRequerida()	public	@Override: devuelve B (aplica cuando no hay piloto).	Microbus
49	boolean	aceptaLicencias(ArrayList<TipoLicencia> licencias)	public	@Override: devuelve true si incluye piloto; si no, usa la validación de Vehiculo.	Microbus
50	int	getUmbralMantenimiento()	public	@Override: devuelve 25.	Microbus
51	String	getCategoria()	public	@Override: devuelve "Microbús".	Microbus
52	String	describirCaracteristicas()	public	@Override: pasajeros y si incluye piloto.	Microbus

53	—	Cliente(String identificador, String nombre, ArrayList<TipoLicencia> licencias)	protected	Constructor: valida identificador y nombre no vacíos y al menos una licencia; conteos en 0.	Cliente
54	String	getIdentificador()	public	Lectura del DPI o NIT.	Cliente
55	String	getNombre()	public	Lectura del nombre.	Cliente
56	ArrayList<TipoLicencia>	getLicencias()	public	Devuelve una copia de las licencias para proteger la lista interna.	Cliente
57	int	getAlquileresConfirmados()	public	Lectura del conteo de alquileres confirmados.	Cliente
58	int	getAlquileresActivos()	public	Lectura del conteo de alquileres activos.	Cliente
59	boolean	puedeConducir(Vehiculo vehiculo)	public	Delega en vehiculo.aceptaLicencias(licencias).	Cliente
60	boolean	alcanzoLimiteActivos()	public	Devuelve true si alquileresActivos $\geq$ getLimiteAlquileresActivos().	Cliente
61	double	calcularDescuento(double subtotal)	public abstract	Descuento según el tipo de cliente.	Cliente
62	int	getLimiteAlquileresActivos()	public abstract	Máximo de alquileres activos simultáneos.	Cliente
63	String	getTipoCliente()	public abstract	Nombre del tipo de cliente para mostrar.	Cliente
64	void	registrarAlquilerConfirmado()	public	Suma uno a confirmados y a activos; solo se llama al confirmar.	Cliente
65	void	registrarAlquilerFinalizado()	public	Resta uno a activos al devolver.	Cliente
66	void	cargarAlquileresPrevios(int cantidad)	public	Registra alquileres históricos (≥ 0) para los datos iniciales.	Cliente
67	String	toString()	public	Sobrescribe Object: tipo, identificador, nombre, licencias y conteos.	Cliente
68	—	ClienteIndividual(String dpi, String nombre, ArrayList<TipoLicencia> licencias)	public	Constructor: valida el DPI con esDpiValido y llama a super.	ClienteIndividual
69	boolean	esDpiValido(String dpi)	public static	Devuelve true si el texto tiene exactamente 13 dígitos.	ClienteIndividual
70	double	calcularDescuento(double subtotal)	public	@Override: 0 si tiene menos de 3 confirmados; 5 % del subtotal a partir del cuarto.	ClienteIndividual

71	int	getLimiteAlquileresActivos()	public	@Override: devuelve 1.	ClienteIndividual
72	String	getTipoCliente()	public	@Override: devuelve "Individual".	ClienteIndividual
73	—	ClienteCorporativo(String nit, String nombreEmpresa, String nombreContacto, ArrayList<TipoLicencia> licencias)	public	Constructor: valida contacto no vacío y llama a super.	ClienteCorporativo
74	String	getNombreContacto()	public	Lectura del contacto.	ClienteCorporativo
75	double	calcularDescuento(double subtotal)	public	@Override: 10 % del subtotal.	ClienteCorporativo
76	int	getLimiteAlquileresActivos()	public	@Override: devuelve 3.	ClienteCorporativo
77	String	getTipoCliente()	public	@Override: devuelve "Corporativo".	ClienteCorporativo
78	String	toString()	public	@Override: agrega el contacto a la descripción.	ClienteCorporativo
79	—	Cotizacion(Vehiculo vehiculo, Cliente cliente, int dias)	public	Constructor: calcula subtotal, descuento y total y reúne las razones de rechazo sin modificar nada.	Cotizacion
80	void	evaluarRazones()	private	Agrega cada razón: vehículo no disponible, licencia inadecuada o límite alcanzado.	Cotizacion
81	boolean	esAlquilable()	public	Devuelve true si no hay razones de rechazo.	Cotizacion
82	ArrayList<String>	getRazonesRechazo()	public	Devuelve una copia de las razones.	Cotizacion
83	Vehiculo / Cliente / int / double	getVehiculo(), getCliente(), getDias(), getSubtotal(), getDescuento(), getTotal()	public	Lecturas usadas para crear el Alquiler.	Cotizacion
84	String	toString()	public	Detalle: características, estado, montos con dos decimales y razones.	Cotizacion
85	—	Alquiler(int numero, Cotizacion cotizacion)	public	Constructor: copia los datos de una cotización válida; queda activo.	Alquiler
86	int / Cliente / Vehiculo / int / double	getNumero(), getCliente(), getVehiculo(), getDias(), getSubtotal(), getDescuento(), getTotal()	public	Lecturas para reportes, historial y devoluciones.	Alquiler
87	boolean	isActive()	public	Indica si el alquiler sigue en curso.	Alquiler
88	void	finalizar()	public	Marca el alquiler como finalizado; no modifica montos.	Alquiler
89	String	toString()	public	Detalle del alquiler con montos a dos decimales.	Alquiler
90	—	RentaMovil()	public	Constructor: colecciones vacías, correlativo en 1, ingresos y descuentos en 0.	RentaMovil

91	void	registrarVehiculo(Vehiculo vehiculo)	public	Agrega a la flota; lanza <code>IllegalArgumentException</code> si la placa ya existe.	RentaMovil
92	void	registrarCliente(Cliente cliente)	public	Agrega a clientes; lanza excepción si el identificador ya existe.	RentaMovil
93	Vehiculo	buscarVehiculo(String placa)	public	Devuelve el vehículo o null si no existe.	RentaMovil
94	Cliente	buscarCliente(String identificador)	public	Devuelve el cliente o null si no existe.	RentaMovil
95	ArrayList<Vehiculo>	getFlota()	public	Copia de la flota para consultas.	RentaMovil
96	ArrayList<Cliente>	getClientes()	public	Copia de los clientes para consultas.	RentaMovil
97	Cotizacion	cotizar(String placa, String identificador, int dias)	public	Valida existencia y días > 0 y devuelve la cotización; no cambia ningún dato.	RentaMovil
98	Alquiler	confirmarAlquiler(Cotizacion cotizacion)	public	Revalida, crea el alquiler, marca el vehículo, actualiza conteos del cliente, ingresos y descuentos.	RentaMovil
99	boolean	registrarDevolucion(String placa)	public	Valida que esté alquilado, finaliza el alquiler, libera al cliente y actualiza el vehículo. Devuelve true si pasó a mantenimiento.	RentaMovil
100	void	finalizarMantenimiento(String placa)	public	Valida que esté en mantenimiento y lo regresa a disponible.	RentaMovil
101	Alquiler	buscarAlquilerActivo(Vehiculo vehiculo)	private	Localiza el alquiler activo del vehículo devuelto.	RentaMovil
102	String	reporteFlota()	public	Cantidad por categoría y por estado.	RentaMovil
103	String	reporteIngresos()	public	Ingresos totales y por categoría, y descuentos otorgados.	RentaMovil
104	ArrayList<Alquiler>	getAlquileresActivos()	public	Lista de alquileres en curso.	RentaMovil
105	ArrayList<Alquiler>	getHistorialCliente(String identificador)	public	Alquileres confirmados del cliente.	RentaMovil
106	double	calcularTotalPagado(String identificador)	public	Suma de los totales pagados por el cliente.	RentaMovil
107	void	main(String[] args)	public static	Punto de entrada: carga datos iniciales y muestra el menú hasta que el usuario sale.	Main
108	void	cargarDatosIniciales()	private static	Registra 2+ vehículos por categoría y 2+ clientes por tipo, sin ingresos.	Main
109	void	mostrarMenu()	private static	Imprime las opciones disponibles.	Main

110	void	registrarVehiculo()	private static	Pide la categoría (única excepción permitida) y construye el objeto correspondiente.	Main
111	void	registrarCliente()	private static	Pide el tipo de cliente y construye el objeto correspondiente.	Main
112	void	consultarFlota() / consultarClientes()	private static	Muestran cada objeto con toString() (polimórfico).	Main
113	void	cotizarAlquiler()	private static	Pide placa, cliente y días, y muestra la cotización.	Main
114	void	alquilarVehiculo()	private static	Muestra la cotización y pregunta si confirma o cancela.	Main
115	void	registrarDevolucion() / finalizarMantenimiento()	private static	Piden la placa y muestran el resultado o el error.	Main
116	void	mostrarReportes()	private static	Muestra los reportes de RentaMovil.	Main
117	int / double / String / boolean	leerEntero(String), leerDouble(String), leerTexto(String), leerSiNo(String)	private static	Lectura segura: repite la pregunta si el formato es incorrecto (try/catch de NumberFormatException).	Main
118	ArrayList<TipoLicencia>	leerLicencias()	private static	Lee una o más licencias válidas separadas por coma.	Main

Herencia y polimorfismo

Justificación de las jerarquías

Lo que tienen en común todos los vehículos (placa, marca, modelo, tarifa, estado, días acumulados, los cambios de estado y la fórmula del subtotal) lo puse una sola vez en Vehiculo, y lo que cambia lo dejé en cada categoría. Con Cliente hice lo mismo: el identificador, el nombre, las licencias y los conteos son comunes, y el descuento y el límite dependen de cada tipo.

Hice abstractas a Vehiculo, VehiculoPasajeros y Cliente porque en el negocio no existe un vehículo sin categoría ni un cliente sin tipo. Si se pudieran crear, no sabrían calcular su recargo, su licencia, su umbral, su descuento ni su límite.

Comportamientos polimórficos

- `calcularRecargo(int)`: es abstracto en `Vehiculo` y lo sobrescriben las cuatro categorías, porque depende de datos que solo tiene cada vehículo. `calcularSubtotal` (que es final) lo manda a llamar.
- `getLicenciaRequerida()` y `aceptaLicencias(...)`: cada categoría dice qué licencia pide. `Microbus` sobrescribe `aceptaLicencias` para no pedir licencia cuando trae piloto.
- `getUmbralMantenimiento()`: cada categoría devuelve su umbral, y `registrarDevolucion` lo usa sin tener que saber qué vehículo es.
- `getCategoria()`, `describirCaracteristicas()` y `toString()`: cada subclase describe sus propios datos, porque solo ella los conoce.
- `calcularDescuento(double)`, `getLimiteAlquileresActivos()` y `getTipoCliente()`: son abstractos en `Cliente` y `ClienteIndividual` y `ClienteCorporativo` ponen sus propias reglas, porque eso depende del tipo y del historial de cada cliente.

Ubicación de la cantidad de pasajeros

Si la ponía en `Vehiculo`, las motos y las camionetas iban a tener un dato que no usan, y si la repetía en `Automovil` y `Microbus` iba a tener el mismo atributo, la misma validación y el mismo getter dos veces. Por eso hice una clase abstracta en medio, `VehiculoPasajeros`, y de ahí heredan `Automovil` y `Microbus`.

Recorrido para confirmar un alquiler

1. Main pide la placa, el cliente y los días, y llama a `empresa.cotizar(...)`. `RentaMovil` busca el `Vehiculo` y el `Cliente` y crea una `Cotizacion`.
2. La `Cotizacion` llama a `vehiculo.calcularSubtotal(dias)`, y ahí se usa el `calcularRecargo` del tipo real del vehículo (polimorfismo).
3. Después llama a `cliente.calcularDescuento(subtotal)`, y se usa la versión del tipo real del cliente (polimorfismo).
4. Revisa las razones: `estaDisponible()`, `puedeConducir(vehiculo)`, que llama a `aceptaLicencias(...)` (polimórfico), y `alcanzoLimiteActivos()`, que usa `getLimiteAlquileresActivos()` (polimórfico).

5. Main enseña la cotización (describirCaracteristicas() es polimórfico) y pregunta si confirma. Si cancela, no cambia nada.
6. Si confirma, RentaMovil.confirmarAlquiler(...) crea el Alquiler con su número, llama a marcarAlquilado() y a registrarAlquilerConfirmado(), y suma el total y el descuento a lo acumulado según getCategory().

Evaluación de la extensión

Para los vehículos eléctricos solo habría que crear VehiculoElectrico, que herede de Vehiculo y sobrescriba el recargo, la licencia, el umbral, la categoría y la descripción. Para los clientes gubernamentales, crear ClienteGubernamental, que herede de Cliente y sobrescriba el descuento, el límite y el tipo. En Main solo se agrega la opción para registrarlos. Vehiculo, Cliente, Cotizacion, Alquiler y RentaMovil no se tocan, porque trabajan con los tipos comunes.

Pruebas

Datos con los que arranca el programa: P-101AAA automóvil automático Q300; P-102AAB automóvil manual Q250 con 28 días acumulados; M-201AAA moto 190 cc Q150; M-202AAB moto 400 cc Q200; C-301AAA camioneta 1.5 t Q200; C-302AAB camioneta 3 t Q350; B-401AAA microbús con piloto Q450; B-402AAB microbús sin piloto Q400. Clientes: Ana López (C, ya tiene 3 alquileres), Luis Pérez (M), María Gómez (B y M), Transportes del Sur (A) y Eventos GT (C).

#	Situación probada	Resultado esperado
1	Cotizar C-301AAA, 3 días, Transportes del Sur	Subtotal Q1,050.00; descuento Q105.00; total Q945.00
2	Cotizar B-401AAA (con piloto), 2 días, Eventos GT (C)	Subtotal Q1,400.00; descuento Q140.00; total Q1,260.00; alquilable sin licencia B
3	Cotizar B-402AAB (sin piloto), 2 días, Eventos GT (C)	Rechazo: licencia inadecuada
4	Cotizar B-402AAB, 2 días, Transportes del Sur (A)	Total Q720.00; alquilable (A autoriza B)
5	Cotizar P-101AAA (automático), 2 días, Ana (3 previos)	Subtotal Q700.00; descuento 5 % Q35.00; total Q665.00

6	Cotizar P-102AAB (manual), 2 días, María	Subtotal y total Q500.00; sin descuento
7	Cotizar M-201AAA (190 cc) y M-202AAB (400 cc), 3 días, Luis	Q450.00 sin recargo; Q675.00 con recargo único de Q75
8	Cotizar P-101AAA con Luis (solo M)	Rechazo: licencia inadecuada
9	Alquilar P-101AAA con Ana y cancelar	Ingresos Q0.00; vehículo Disponible; Ana sigue con 3 confirmados
10	Alquilar P-101AAA con Ana y confirmar	Alquiler #1 por Q665.00; vehículo Alquilado; Ana con 4 confirmados y 1 activo
11	Ana intenta otro alquiler	Rechazo: límite de alquileres activos; nada cambia
12	Cotizar P-101AAA (alquilado)	Muestra montos y razón "no disponible"; ingresos sin cambio
13	Transportes del Sur intenta un cuarto alquiler activo	Rechazo: límite de 3 alquileres activos
14	Devolver un vehículo disponible	Rechazo: el vehículo no está alquilado
15	María alquila P-102AAB 2 días y lo devuelve	$28 + 2 = 30$: pasa a mantenimiento; ingresos no cambian al devolver
16	Finalizar mantenimiento de P-102AAB	Disponible con acumulado 0
17	Finalizar mantenimiento de un vehículo que no está en mantenimiento	Rechazo con mensaje
18	Placa o cliente inexistente	Rechazo con mensaje
19	Placa repetida, tarifa 0, DPI de 12 dígitos o cliente sin licencias	Rechazo; no se registra
20	Ingresar "abc" o 0 días	Mensaje de error; el programa no termina
21	Reportes finales	Conteos, ingresos y descuentos coinciden solo con los alquileres confirmados